Lab Report

Cloud Computing

Kubernetes, Hands-On

60-Minute Lab: Build, Deploy, Scale, Update & Debug

Name: Neel Prajapati

Enrollment No.: 202611022

Subject: Cloud Computing

Lab URL: https://k8slabs.learnwithdutta.com/

Date: 16 October 2026

Contents

Introduction **2**

Setup — Building the Cluster **4**

1 Look at Your Cluster **6**
1.1 Checking the Current Context and Nodes . . . 6
1.2 Why Apps Never Run on the Control Plane . . . 6
1.3 System Pods in kube-system . . . 7

2 Run Your First App **8**
2.1 Creating the Deployment YAML . . . 8
2.2 Applying the Deployment . . . 9
2.3 One File, Three Objects . . . 9

3 Open it in a Browser **10**
3.1 Creating the Service YAML . . . 10
3.2 Applying the Service and Checking Endpoints . . . 10
3.3 Visiting the App in the Browser . . . 11
3.4 Port Mapping Summary . . . 11
3.5 Quick Tunnel Alternative . . . 12

4 Scale It, Then Kill It **13**
4.1 Scale Up to 5 Replicas . . . 13
4.2 Scale Back Down to 3 . . . 13
4.3 Deliberately Deleting a Pod . . . 13

5 Update, Redeploy, Undo **15**
5.1 5.1 — Imperative Update . . . 15
5.2 5.2 — Declarative Update . . . 15
5.3 5.3 — Rollback . . . 16
5.4 5.4 — Idempotent Apply (Nothing Changed) . . . 16

6 Break It and Fix It **17**
6.1 6.1 — Deliberately Break the Deployment . . . 17
6.2 6.2 — Diagnose with `describe` . . . 17
6.3 6.3 — Try `logs` (and understand why it fails here) . . . 19
6.4 6.4 — Fix It . . . 19

Bonus: Declarative Image Update Result **20**

Cleanup **21**

Summary and Conclusion **22**

Introduction

Objective

This report documents the hands-on Kubernetes 60-minute lab performed on 16 October 2026. The goals were to:

1. Build a 3-node Kubernetes cluster on a local machine using `kind`.
2. Inspect cluster components and understand control-plane vs. worker node roles.
3. Deploy the first application (nginx) using a Deployment YAML manifest.
4. Expose the application on a real browser port using a NodePort Service.
5. Scale up/down replicas and observe self-healing behaviour.
6. Update the application image and perform a rollback.
7. Deliberately break the cluster and diagnose the error.

What is Kubernetes?

Kubernetes is **a program you give a wish to. It then works non-stop to make that wish true.**

Unlike Docker (which executes a single `docker run` command), Kubernetes accepts a **standing instruction**: "keep three copies of this app running". It continuously monitors the cluster and repairs deviations — analogous to a thermostat set to a fixed temperature.

Key Terminology

Term	Meaning
Cluster	A group of machines managed as one unit.
Node	One machine (here: a Docker container acting as a server).
Control Plane	The master node that stores configuration and schedules Pods.
Worker	A node that actually runs application Pods.
Pod	One running copy of an app — disposable, replaced frequently.
Deployment	The instruction: "keep N copies of this image running".
Service	One stable address in front of changing Pods, matched by labels.

Lab Plan

Time	Step	Achievement
0–10 min	Setup	Built a Kubernetes cluster on a laptop
10–15 min	1. Look at cluster	Understood the three-node architecture
15–26 min	2. Run first app	Wrote one YAML; Kubernetes spread it on two machines
26–37 min	3. Open in browser	App accessible on a real port (30000)
37–44 min	4. Scale & kill	Deleted running app; nothing went down
44–52 min	5. Update & undo	Shipped new version and rolled it back
52–60 min	6. Break & fix	Read an error and found the root cause

Setup — Building the Cluster

Tool Verification

```
$ docker ps
$ kubectl version --client
$ kind version
```

Working Directory

```
$ mkdir -p ~/k8s-lab && cd ~/k8s-lab
```

Generate Configuration and Build the Cluster

The shell block below writes `kind-config.yaml` (one control-plane, two workers, ports 30000–30050 mapped for browser access), then creates the cluster:

```
$ {
  echo 'kind: Cluster'
  echo 'apiVersion: kind.x-k8s.io/v1alpha4'
  echo 'name: k8s-lab'
  echo 'nodes:'
  echo '  - role: control-plane'
  echo '    extraPortMappings:'
  for p in $(seq 30000 30050); do
    echo "      - containerPort: $p"
    echo "        hostPort: $p"
    echo "        protocol: TCP"
  done
  echo '  - role: worker'
  echo '  - role: worker'
} > kind-config.yaml

$ kind create cluster --config kind-config.yaml
```

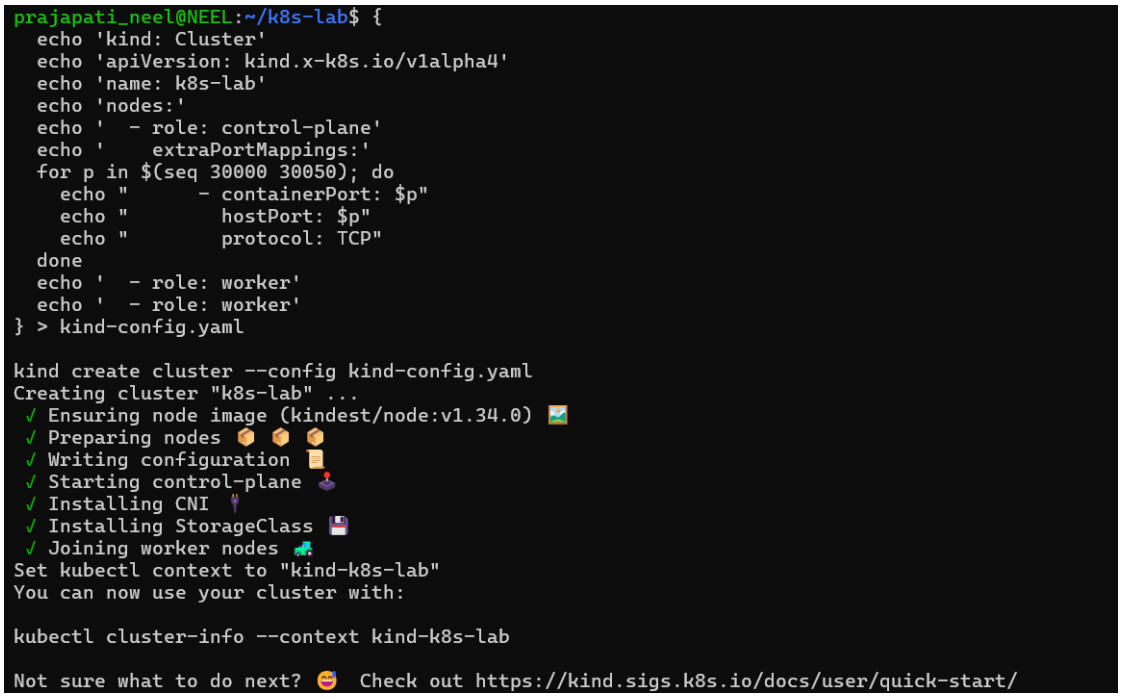


Figure 1: Setup — Creating the kind cluster. The configuration is written and `kind create cluster` runs successfully, completing all phases: Ensuring node image, Preparing nodes, Writing configuration, Starting control-plane, Installing CNI, Installing StorageClass, and Joining worker nodes.

Verify Nodes

```
$ kubectl get nodes
$ docker ps --format "{{.Names}}"
```

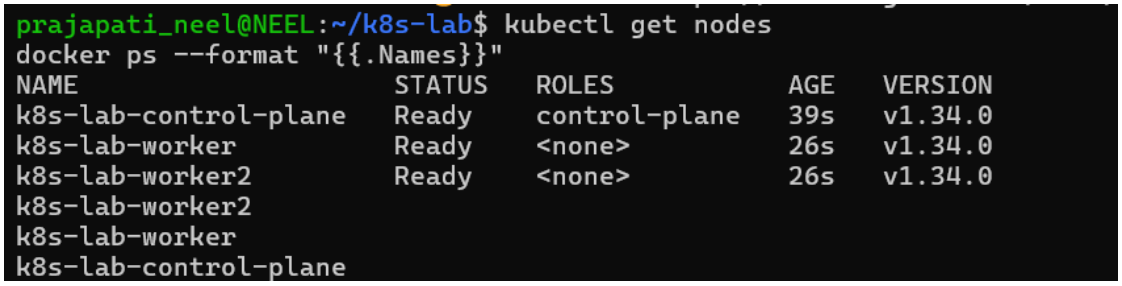


Figure 2: Setup — All three nodes (`k8s-lab-control-plane`, `k8s-lab-worker`, `k8s-lab-worker2`) are `Ready` running Kubernetes v1.34.0.

Explanation

kind = "Kubernetes IN Docker". It runs a full cluster inside containers on a laptop. Cleanup is simple: deleting the cluster means deleting three Docker containers.

Step 1

Look at Your Cluster

Goal: Meet the three machines and see Kubernetes running itself. **~5 min | Easy**

1.1 Checking the Current Context and Nodes

```
$ kubectl config current-context
$ kubectl get nodes -o wide
$ kubectl get pods
```

```
prajapati_neel@NEEL:~/k8s-lab$ kubectl config current-context
kubectl get nodes -o wide
kubectl get pods
kind-k8s-lab
NAME                  STATUS   ROLES           AGE   VERSION   INTERNAL-IP   EXTERNAL-IP
      CONTAINER-RUNTIME
k8s-lab-control-plane   Ready    control-plane   50s   v1.34.0   172.18.0.4    <none>
SL2   containerd://2.1.3
k8s-lab-worker          Ready    <none>          37s   v1.34.0   172.18.0.3    <none>
SL2   containerd://2.1.3
k8s-lab-worker2         Ready    <none>          37s   v1.34.0   172.18.0.2    <none>
SL2   containerd://2.1.3
No resources found in default namespace.
```

Figure 1.1: Step 1 — Current context is `kind-k8s-lab`. Nodes listed with internal IPs and container runtime (containerd 2.1.3). `kubectl get pods` returns "No resources found in default namespace" — correct, nothing deployed yet.

Explanation

"No resources found" is **correct, not an error.** Always verify your context (`kubectl config current-context`) before running destructive commands.

1.2 Why Apps Never Run on the Control Plane

```
$ kubectl describe node k8s-lab-control-plane | grep -i -A1 taints
```

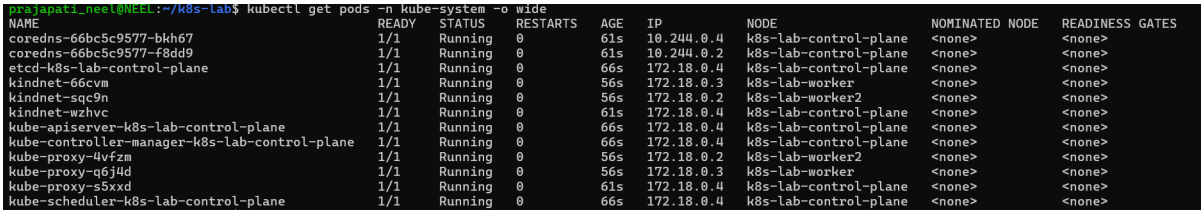
Expected Output

```
Taints: node-role.kubernetes.io/control-plane:NoSchedule
```

A **taint** is a "keep out" sign. Without a matching toleration in the Pod spec, the scheduler never places application Pods on the control plane — preventing a rogue app from consuming control-plane resources.

1.3 System Pods in kube-system

```
$ kubectl get pods -n kube-system -o wide
```



```
prajapati_neel@NEEL:~/k8s-lab$ kubectl get pods -n kube-system -o wide
NAME                                          READY   STATUS    RESTARTS   AGE   IP           NODE                  NOMINATED NODE   READINESS GATES
coredns-66bc5c9577-bkh67                      1/1     Running   0          61s   10.244.0.4   k8s-lab-control-plane   <none>           <none>
coredns-66bc5c9577-f8dd9                      1/1     Running   0          61s   10.244.0.2   k8s-lab-control-plane   <none>           <none>
etcd-k8s-lab-control-plane                    1/1     Running   0          66s   172.18.0.4   k8s-lab-control-plane   <none>           <none>
kindnet-66cvm                                 1/1     Running   0          56s   172.18.0.3   k8s-lab-worker          <none>           <none>
kindnet-sqc9n                                 1/1     Running   0          56s   172.18.0.2   k8s-lab-worker2         <none>           <none>
kindnet-wzhvc                                 1/1     Running   0          61s   172.18.0.4   k8s-lab-control-plane   <none>           <none>
kube-apiserver-k8s-lab-control-plane          1/1     Running   0          66s   172.18.0.4   k8s-lab-control-plane   <none>           <none>
kube-controller-manager-k8s-lab-control-plane 1/1     Running   0          66s   172.18.0.4   k8s-lab-control-plane   <none>           <none>
kube-proxy-4vfzm                              1/1     Running   0          56s   172.18.0.2   k8s-lab-worker2         <none>           <none>
kube-proxy-q6j4d                              1/1     Running   0          56s   172.18.0.3   k8s-lab-worker          <none>           <none>
kube-proxy-s5xxd                              1/1     Running   0          61s   172.18.0.4   k8s-lab-control-plane   <none>           <none>
kube-scheduler-k8s-lab-control-plane          1/1     Running   0          66s   172.18.0.4   k8s-lab-control-plane   <none>           <none>
```

Figure 1.2: Step 1 — System pods in the `kube-system` namespace. All are `Running` with 0 restarts. Control-plane pods (`etcd`, `kube-apiserver`, `kube-controller-manager`, `kube-scheduler`, `coredns`) run only on the control-plane node. Networking pods (`kube-proxy`, `kindnet`) run on all three nodes.

Pod	Where	Job
`kube-apiserver`	control plane	Front door — all commands go here
`etcd`	control plane	The notebook — stores everything
`kube-scheduler`	control plane	Chooses which node runs each new Pod
`kube-controller-manager`	control plane	The thermostat — desired vs. actual
`coredns`	control plane	DNS resolution inside the cluster
`kube-proxy, kindnet`	all three nodes	Networking (one copy per node)

Step 2

Run Your First App

Goal: Write one file. Watch Kubernetes build three objects and place them on two machines. **~11 min | The main one**

2.1 Creating the Deployment YAML

```
$ nano deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:stable
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
```

Field	Meaning
`kind: Deployment`	Type of wish: keep copies of an app running.
`replicas: 3`	**The dial.** Three copies, always.
`app: nginx` (twice)	Label sticker. Deployment and Service both use it.
`template`	Cookie-cutter — describes one Pod.
`image: nginx:stable`	Which container image to pull.
`requests`	CPU/memory reservation. `50m` = 5% of one core.

2.2 Applying the Deployment

```
$ kubectl apply --dry-run=client -f deployment.yaml
$ kubectl apply -f deployment.yaml
$ kubectl wait --for=condition=Available deployment/nginx --timeout
    =120s
$ kubectl get pods -o wide
```

```
prajapati_neel@NEEL:~/k8s-lab$ nano deployment.yaml
prajapati_neel@NEEL:~/k8s-lab$ kubectl apply --dry-run=client -f deployment.yaml
kubectl apply -f deployment.yaml
kubectl wait --for=condition=Available deployment/nginx --timeout=120s
kubectl get pods -o wide
deployment.apps/nginx created (dry run)
deployment.apps/nginx created
deployment.apps/nginx condition met
NAME                     READY   STATUS    RESTARTS   AGE   IP           NODE              NOMINATED NODE   READINESS GATES
nginx-676775dcc5-h9t66   1/1     Running   0          29s   10.244.2.2   k8s-lab-worker    <none>           <none>
nginx-676775dcc5-nbj8k   1/1     Running   0          29s   10.244.1.3   k8s-lab-worker2   <none>           <none>
nginx-676775dcc5-szlfw   1/1     Running   0          29s   10.244.1.2   k8s-lab-worker2   <none>           <none>
```

Figure 2.1: Step 2 — Dry-run passes, deployment created, and `kubectl wait` confirms `condition met`. Three nginx pods (`nginx-676775dcc5-h9t66`, `-nbj8k`, `-szlfw`) are `Running` across both worker nodes.

Explanation

Look at the NODE column. You never chose a machine; the *scheduler* decided. The Pods are spread across `k8s-lab-worker` and `k8s-lab-worker2` automatically.

2.3 One File, Three Objects

```
$ kubectl get deployments
$ kubectl get replicasets
$ kubectl get pods
```

- **Deployment** — what you wrote; manages rolling updates.
- **ReplicaSet** — created for you; ensures exactly 3 Pods exist at all times.
- **Pod** — created by the ReplicaSet; one running copy with its own IP.

You created **one** object. Kubernetes built the other two and maintains them forever.

Step 3

Open it in a Browser

Goal: Give three moving Pods one fixed address, then reach it on a real port. **~11 min | Easy | The payoff**

3.1 Creating the Service YAML

```
$ nano service.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80          # the Service's port inside the cluster
      targetPort: 80    # the port inside the container
      nodePort: 30000   # the port you type in your browser
```

3.2 Applying the Service and Checking Endpoints

```
$ kubectl apply -f service.yaml
$ kubectl get services
$ kubectl get endpoints nginx
```

```
prajapati_neel@NEEL:~/k8s-lab$ nano service.yaml
prajapati_neel@NEEL:~/k8s-lab$ kubectl apply -f service.yaml
kubectl get services
kubectl get endpoints nginx
service/nginx created
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP        3m46s
nginx        NodePort    10.96.151.172   <none>        80:30000/TCP   0s
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME    ENDPOINTS                                      AGE
nginx   10.244.1.2:80,10.244.1.3:80,10.244.2.2:80   0s
```

Figure 3.1: Step 3 — Service applied. `kubectl get services` shows the nginx NodePort Service on `80:30000/TCP`. `kubectl get endpoints nginx` lists three endpoints (one per Pod) at their respective IPs — proof that label matching worked.

Warning

`selector: app: nginx` must exactly match the label in `deployment.yaml`. A mismatch produces zero endpoints and silent failures — the most common Kubernetes bug.

3.3 Visiting the App in the Browser

Navigate to **`http://localhost:30000`** in a browser:

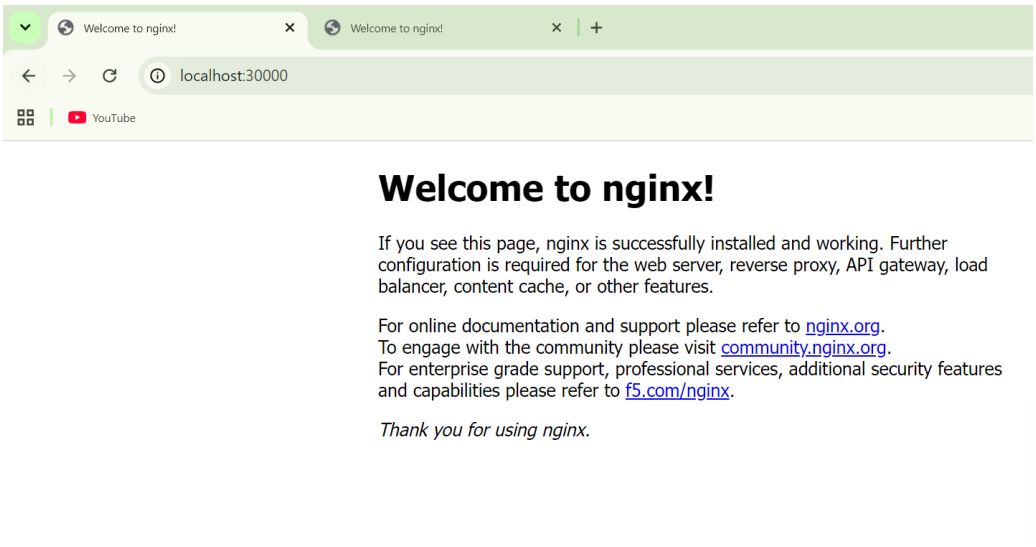


Figure 3.2: Step 3 — The **"Welcome to nginx!"** page is displayed at `localhost:30000`. The NodePort Service is forwarding traffic from the control-plane node through to a running Pod on a worker node.

3.4 Port Mapping Summary

Field	Lives on	Purpose
`nodePort: 30000`	every node	External port; must be 30000–32767.
`port: 80`	the Service	Internal cluster port for other Pods.
`targetPort: 80`	the Pod	Port the container actually listens on.

3.5 Quick Tunnel Alternative

```
$ kubectl port-forward service/nginx 8080:80
```

```
prajapati_neel@NEEL:~/k8s-lab$ kubectl port-forward service/nginx 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
prajapati_neel@NEEL:~/k8s-lab$ kubectl scale deployment nginx --replicas=5=5
kubectl get pods -o wide
kubectl get endpoints nginx
deployment.apps/nginx scaled
NAME                     READY   STATUS              RESTARTS   AGE     IP           NODE              NOMINATED NODE
nginx-676775dcc5-4nhvm   0/1     ContainerCreating   0          0s      <none>       k8s-lab-worker2   <none>
nginx-676775dcc5-d4f5v   0/1     ContainerCreating   0          0s      <none>       k8s-lab-worker    <none>
nginx-676775dcc5-h9t66   1/1     Running             0          6m48s   10.244.2.2   k8s-lab-worker    <none>
nginx-676775dcc5-nbj8k   1/1     Running             0          6m48s   10.244.1.3   k8s-lab-worker2   <none>
nginx-676775dcc5-szlfw   1/1     Running             0          6m48s   10.244.1.2   k8s-lab-worker2   <none>
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME    ENDPOINTS                                   AGE
nginx   10.244.1.2:80,10.244.1.3:80,10.244.2.2:80   5m15s
```

Figure 3.3: Step 3 / 4 combined — `kubectl port-forward` active in one terminal while `kubectl scale -replicas=5` is run in another. The scaling shows two new pods in `ContainerCreating`. Endpoints still show only the original three until new pods become Ready.

Step 4

Scale It, Then Kill It

Goal: Turn the dial up, then delete a running Pod and watch nothing bad happen. **~7 min | Easy | The fun one**

4.1 Scale Up to 5 Replicas

```
$ kubectl scale deployment nginx --replicas=5
$ kubectl get pods -o wide
$ kubectl get endpoints nginx
```

Two new Pods appeared. The original three were **not restarted**. The Service automatically picked up the new Pods by label — `service.yaml` was untouched.

4.2 Scale Back Down to 3

```
$ kubectl scale deployment nginx --replicas=3
$ kubectl get pods -o wide -w
```

```
prajapati_neel@NEEL:~/k8s-lab$ kubectl scale deployment nginx --replicas=3
deployment.apps/nginx scaled
prajapati_neel@NEEL:~/k8s-lab$ kubectl get pods -o wide -w
NAME                     READY   STATUS    RESTARTS   AGE     IP           NODE              NOMINATED NODE   READINESS GATES
nginx-676775dcc5-d4f5v   1/1     Running   0          2m4s    10.244.2.3   k8s-lab-worker    <none>           <none>
nginx-676775dcc5-h9t66   1/1     Running   0          8m52s   10.244.2.2   k8s-lab-worker    <none>           <none>
nginx-676775dcc5-szlfw   1/1     Running   0          8m52s   10.244.1.2   k8s-lab-worker2   <none>           <none>
```

Figure 4.1: Step 4 — After scaling back to 3. All three Pods are `Running` and distributed across both worker nodes (`k8s-lab-worker` and `k8s-lab-worker2`).

4.3 Deliberately Deleting a Pod

```
$ kubectl get pods
```

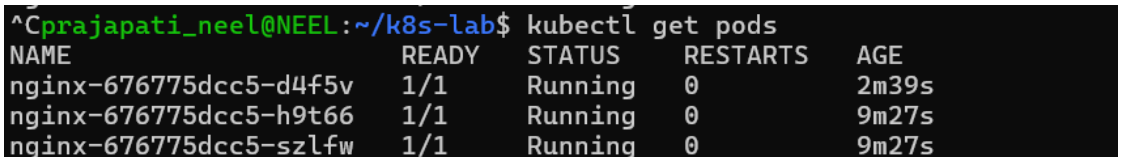


Figure 4.2: Step 4 — Three pods listed before deletion. Pod `nginx-676775dcc5-szlfw` was selected for deletion to demonstrate self-healing.

```
$ kubectl delete pod nginx-676775dcc5-szlfw
```

Within approximately 2 seconds a new Pod with a new name and new IP appears, placed by the scheduler on whichever node has capacity. The site never went down — the two surviving Pods served all requests during the replacement.

Explanation

That is the thermostat working. The ReplicaSet counted 2 Pods, wanted 3, and created one more. This is why Pods are called **disposable** — never write a Pod IP address into application code. The Service exists precisely because Pod IPs always change.

Warning

To really remove the app, change the *wish*: `kubectl delete deployment nginx`. Deleting individual Pods only causes the ReplicaSet to create replacements.

Step 5

Update, Redeploy, Undo

Goal: Ship a new version with zero downtime, force a redeploy, then take it all back. **~8 min | Medium**

5.1 5.1 — Imperative Update

```
$ kubectl set image deployment/nginx nginx=nginx:stable-alpine
$ kubectl rollout status deployment/nginx
```

```
prajapati_neel@NEEL:~/k8s-lab$ kubectl delete pod nginx-676775dcc5-szlfw
pod "nginx-676775dcc5-szlfw" deleted from default namespace
prajapati_neel@NEEL:~/k8s-lab$ kubectl set image deployment/nginx nginx=nginx:stable-alpine
kubectl rollout status deployment/nginx
deployment.apps/nginx image updated
Waiting for deployment "nginx" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "nginx" rollout to finish: 1 old replicas are pending termination...
deployment "nginx" successfully rolled out
```

Figure 5.1: Step 5.1 — Imperative image update to `nginx:stable-alpine`. Rolling update: 1 new replica updated, then 2, then old replicas terminate. Final message: "deployment `nginx` successfully rolled out". The site remained accessible throughout.

5.2 5.2 — Declarative Update

Edit `deployment.yaml` to change the image and resource requests:

```
$ nano deployment.yaml      # change image to burgxy/webapps:v4
$ kubectl apply -f deployment.yaml
$ kubectl rollout status deployment/nginx
```

```
prajapati_neel@NEEL:~/k8s-lab$ nano deployment.yaml
prajapati_neel@NEEL:~/k8s-lab$ kubectl apply -f deployment.yaml
kubectl rollout status deployment/nginx
deployment.apps/nginx configured
Waiting for deployment "nginx" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "nginx" rollout to finish: 1 old replicas are pending termination...
deployment "nginx" successfully rolled out
```

Figure 5.2: Step 5.2 — Declarative `kubectl apply -f deployment.yaml`. Status shows `configured` and the rolling update completes successfully. This approach is standard in production — changes live in Git and are reproducible.

After the update, `kubectl get replicasets` shows **two ReplicaSets**: the new one is active; the old one is empty but retained as the **undo button**.

5.3 5.3 — Rollback

```
$ kubectl rollout undo deployment/nginx
$ kubectl rollout status deployment/nginx
```

5.4 5.4 — Idempotent Apply (Nothing Changed)

```
$ kubectl apply -f deployment.yaml
```

```
prajapati_neel@NEEL:~/k8s-lab$ kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
nginx-5587d869bc-8mc2g   1/1     Running   0          56s
nginx-5587d869bc-cxgb7   1/1     Running   0          19s
nginx-5587d869bc-v2l6g   1/1     Running   0          35s
prajapati_neel@NEEL:~/k8s-lab$  kubectl apply -f deployment.yaml
deployment.apps/nginx unchanged
```

Figure 5.3: Step 5.4 — After the update, three pods from the new ReplicaSet (`nginx-5587d869bc-*`) are `Running`. Applying the same unchanged YAML returns `deployment.apps/nginx unchanged` — `apply` is fully idempotent.

Warning

Do not redeploy by deleting all Pods. `kubectl delete pod -l app=nginx` removes every copy at once — users see errors for seconds. Use `kubectl rollout restart deployment/nginx` instead; it replaces Pods one at a time with zero downtime.

Step 6

Break It and Fix It

Goal: Cause a real failure, find the cause in 30 seconds, and repair it. **~8 min | The useful one**

6.1 6.1 — Deliberately Break the Deployment

```
$ kubectl set image deployment/nginx nginx=nginx:this-tag-does-not-
    exist
```

Wait 20 seconds:

```
$ kubectl get pods
$ curl -s http://localhost:30000 | head -n 1
```

Explanation

The old Pods kept running and kept serving. The update *stopped* instead of destroying a working app. **A failed deployment is not automatically an outage.**

6.2 6.2 — Diagnose with `describe`

```
$ kubectl describe pod <broken-pod-name>
```

```
prajapati_neel@NEEL:~/k8s-lab$ kubektl describe pod nginx-85f8d6bc4f-n556t
kubektl: command not found
prajapati_neel@NEEL:~/k8s-lab$ kubectl describe pod nginx-85f8d6bc4f-n556t
Name:             nginx-85f8d6bc4f-n556t
Namespace:        default
Priority:         0
Service Account:  default
Node:             k8s-lab-worker/172.18.0.3
Start Time:       Fri, 16 Oct 2026 12:12:10 +0000
Labels:           app=nginx
                  pod-template-hash=85f8d6bc4f
Annotations:      <none>
Status:           Running
IP:               10.244.2.5
IPs:
  IP:           10.244.2.5
Controlled By:  ReplicaSet/nginx-85f8d6bc4f
Containers:
  nginx:
    Container ID:   containerd://0d69ef18fdace0559d688ecbc5e1f4686fe0c968102a1978826e7b31e02a5327
    Image:          caddy
    Image ID:       docker.io/library/caddy@sha256:685ae3d7c34d9b5f8caeddc5d14051db7bbd88ff1f5a2fa0c16bd8ca328fb7af
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Fri, 16 Oct 2026 12:12:37 +0000
    Ready:          True
    Restart Count:  0
    Requests:
      cpu:        50m
      memory:     64Mi
```

Figure 6.1: Step 6 — `kubectl describe pod` on the broken pod. Full metadata visible: Pod assigned to `k8s-lab-worker`, Status `Running`, Labels, Container details (image: caddy, Port: 80/TCP). Events section at bottom shows the full lifecycle trace.

```
  nginx:
    Container ID:   containerd://0d69ef18fdace0559d688ecbc5e1f4686fe0c968102a1978826e7b31e02a5327
    Image:          caddy
    Image ID:       docker.io/library/caddy@sha256:685ae3d7c34d9b5f8caeddc5d14051db7bbd88ff1f5a2fa0c16bd8ca328fb7af
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Fri, 16 Oct 2026 12:12:37 +0000
    Ready:          True
    Restart Count:  0
    Requests:
      cpu:        50m
      memory:     64Mi
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-fjfs9 (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-fjfs9:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    Burstable
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  116s  default-scheduler  Successfully assigned default/nginx-85f8d6bc4f-n556t to k8s-lab-worker
  Normal  Pulling    116s  kubelet            spec.containers{nginx}: Pulling image "caddy"
  Normal  Pulled     89s   kubelet            spec.containers{nginx}: Successfully pulled image "caddy" in 26.453s (26
24886479 bytes.
```

Figure 6.2: Step 6 — Continuation of `kubectl describe pod`. Container conditions all `True`, Volumes, QoS Class (`Burstable`), Tolerations, and Events: Scheduled → Pulling → Pulled (26s) → Running. For a broken pod the Events section would instead show `Failed` and `ImagePullBackOff`.

The Events section gives the answer in plain English:

1. `Scheduled` — scheduler placed the Pod successfully.
2. `Pulling` — kubelet started pulling the image.
3. `Failed` — image not found (wrong tag).
4. `ImagePullBackOff` — Kubernetes backs off, retrying with increasing delays.

6.3 6.3 — Try `logs` (and understand why it fails here)

```
$ kubectl logs <broken-pod-name>
```

Expected Output

```
Error from server (BadRequest): container "nginx" is waiting to
    start:
trying and failing to pull image
```

Explanation

The debugging rule:
Use **`describe`** when the container *never started* (image issues, scheduling failures).
Use **`logs`** when the container *started but behaves wrongly* (application crashes, bad config).

6.4 6.4 — Fix It

```
$ kubectl rollout undo deployment/nginx
$ kubectl rollout status deployment/nginx
$ kubectl get pods -o wide
$ kubectl get endpoints nginx
```

Success criteria: 3/3 Pods `Running`, three endpoints, and `http://localhost:30000` still serving the nginx page.

Status	Root Cause	First Command
`Pending`	No node had sufficient resources	`kubectl describe pod NAME`
`ContainerCreating`	Image being pulled (normal)	Wait; if stuck, `describe`
`ImagePullBackOff`	Wrong image name or tag	`kubectl describe pod NAME`
`CrashLoopBackOff`	App starts then crashes	`kubectl logs NAME -previous`
`Running` but 0/1	Alive, not passing readiness	`kubectl describe pod NAME`
`Running`, user errors	Label mismatch, no endpoints	`kubectl get endpoints NAME`

Bonus: Declarative Image Update Result

During Step 5's declarative update, the image was changed to use the **Caddy** web server. After a successful zero-downtime rolling update, the browser confirmed the new application:

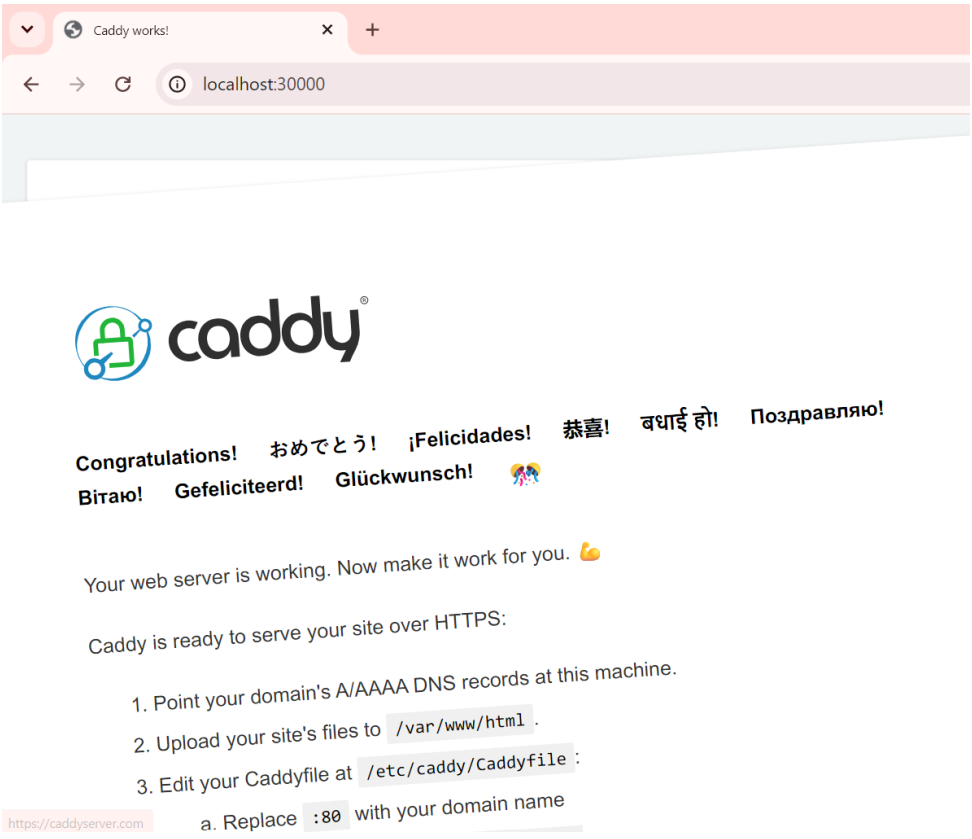


Figure 6.3: Caddy web server running at `localhost:30000` after the rolling update. The page reads "Your web server is working. Now make it work for you." — confirming the Kubernetes rolling update mechanism replaced all Pods without downtime.

Cleanup

Option A — Remove the App, Keep the Cluster

```
$ kubectl delete -f service.yaml
$ kubectl delete -f deployment.yaml
$ kubectl get all
```

Only the built-in `kubernetes` Service remains.

Option B — Delete the Entire Cluster

```
$ kind delete cluster --name k8s-lab
$ kind get clusters      # prints nothing
```

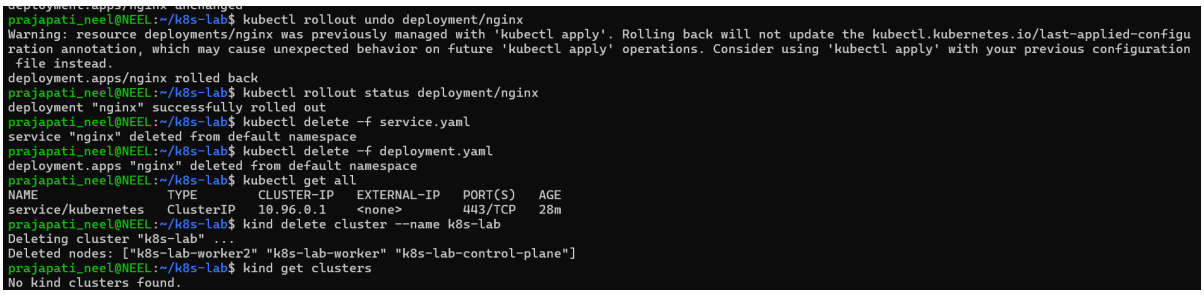


Figure 6.4: Cleanup — `kubectl rollout undo` completes the rollback, then `kubectl delete -f service.yaml` and `kubectl delete -f deployment.yaml` remove the app. `kubectl get all` confirms only the built-in `service/kubernetes` remains. Finally, `kind delete cluster -name k8s-lab` deletes all three container nodes and `kind get clusters` returns nothing.

Summary and Conclusion

Six Things Learned

1. **Kubernetes is a thermostat for software.** Write the desired state; Kubernetes continuously reconciles reality to match it.
2. **One control plane, two workers.** A taint keeps apps off the control plane, preventing a rogue app from making the cluster unmanageable.
3. **One YAML → three objects.** A single Deployment produced a Deployment, a ReplicaSet, and three Pods on two machines never explicitly named. `created` means "wish saved", not "app running".
4. **Services find Pods by label.** A Service is a stable address in front of ephemeral Pods. `kubectl get endpoints` is the proof labels matched.
5. **Deleting a Pod does not delete an app.** Only changing the desired state does. Use `rollout restart` for safe, zero-downtime Pod replacement.
6. **Debugging rule:** `describe` when it never started; `logs` when it started but behaves wrongly. A failed update is not automatically an outage.

Key Commands Reference

Goal	Command
Check which cluster you are on (always do first!)	`kubectl config current-context`
See everything created	`kubectl get all`
See Pods with IPs and nodes	`kubectl get pods -o wide`
Watch changes live	`kubectl get pods -w`
Find why something is unhappy	`kubectl describe pod NAME`
Read app output	`kubectl logs NAME`
Check a Service found its Pods	`kubectl get endpoints NAME`
Change the number of copies	`kubectl scale deployment NAME -replicas=N`
Ship a new image	`kubectl set image deployment/NAME c=img:ta`
Replace all Pods safely	`kubectl rollout restart deployment/NAME`
Undo the last change	`kubectl rollout undo deployment/NAME`
Validate YAML before applying	`kubectl apply -dry-run=client -f file.yaml`

Conclusion

This 60-minute lab provided full hands-on experience with core Kubernetes workflows used in real production environments: cluster creation, declarative deployment, service exposure, horizontal scaling, rolling updates, and fault diagnosis.

The key insight is the **declarative model**: express *what* you want, not *how* to achieve it. Kubernetes then continuously reconciles actual state with desired state, delivering self-healing, zero-downtime deployments, and safe rollbacks as built-in features — not scripts to maintain.